

Programación paralela

Trabajo práctico final

2025

Castro, Sofía
Potinski, Mijail Andrés

Profesor Alonso, Martín

Consigna

EDGE-DETECTING:

Partiendo del código de ejemplo en: Assignment_3/ex_1/hw3_ex1.cu [github](#)
Implementar los 3 pasos (grayscale, gauss filter y sobel filter)

Deben realizarse 4 implementaciones:

- * CPU serie
- * CPU con openMP
- * CUDA con global memory
- * CUDA con shared memory

Analizar y documentar las diferencias de performance entre las 4 implementaciones

MPI:

Paralelizar con MPI el código de mergesort.c

1. randomize() Se ejecuta en el nodo 0 y se reparten los datos equitativamente entre el total de nodos.
2. Cada nodo calcula el histograma de sus datos.
3. Cada nodo ordena sus datos y luego se juntan y ordenan con merge().

[LINK GITHUB CON LAS IMPLEMENTACIONES](#)

Introducción

La diferencia en el tiempo de ejecución entre un programa serie y su versión paralela puede ser significativa si se utiliza correctamente; de lo contrario, el paralelismo podría incluso degradar el rendimiento. Para lograr mejoras efectivas, es fundamental identificar qué tareas pueden paralelizarse y cuáles dependen de resultados previos. En este análisis se consideran distintos tipos de dependencias, como flujo, antidependencias, salida, entrada/salida y control.

En el presente trabajo se toma un código para la detección de bordes en imágenes, se analiza mediante herramientas de profiling y luego se implementan mejoras de rendimiento, con el objetivo de reducir el tiempo de ejecución y aprovechar de manera óptima los recursos computacionales.

Para ello, se prueban diferentes metodologías de paralelismo: con CPU mediante OpenMP, y con GPU utilizando CUDA con memoria global y shared memory.

Además, se evalúa la eficiencia del paralelismo mediante paso de mensajes (MPI), aplicándolo a la mejora de un código serie del algoritmo de ordenamiento mergesort.

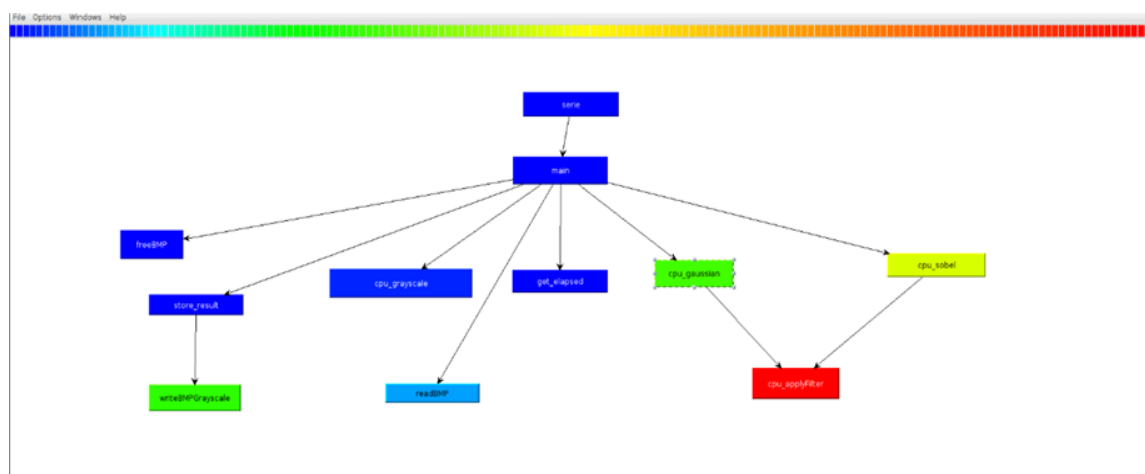
PARTE 1: Edge-Detecting

A continuación, se describe la primera parte del trabajo, centrada en la detección de bordes en imágenes. Se detallan los tres pasos principales del algoritmo —conversión a escala de grises, filtrado gaussiano y aplicación del filtro de Sobel—, así como las cuatro implementaciones realizadas: CPU en serie, CPU paralelizada con OpenMP, GPU con memoria global y GPU con memoria compartida. Se analizan tanto las estrategias de paralelización empleadas como los resultados obtenidos, con el fin de evaluar las mejoras de rendimiento y el aprovechamiento de los recursos computacionales en cada caso.

Desarrollo

Para empezar el análisis del programa base, se utilizó una herramienta de profiling llamada **score-p**. Esta permite analizar el rendimiento del programa por medio de métricas de tiempo de ejecución de los diferentes métodos del programa y el uso de los recursos del sistema. Posibilita la identificación de posibles cuellos de botella, sobrecargas de métodos o inclusive el no aprovechamiento de los recursos en forma óptima.

Una vez que se obtuvo la información de profiling, se prosiguió a hacer uso del framework **TAU** usado en HPC (High Performance Computing) para instrumentación, análisis y perfilado de aplicaciones paralelas y su visualizador gráfico **Paraprof** para tener una mayor claridad de los resultados.



TAU ParaProf Statistics for: node 0 - scorep-20250718_2225_1663323204023/profile.cubex

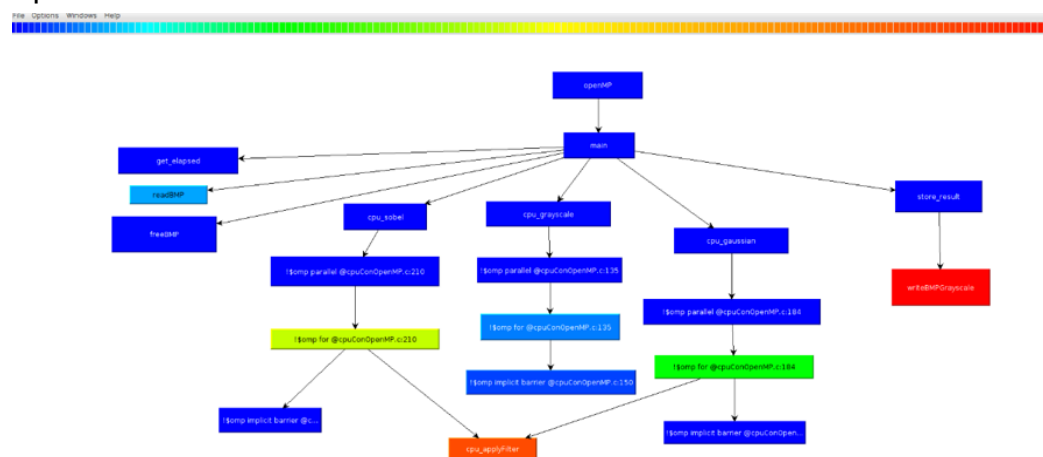
Name	Exclusive Time	Inclusive Time	Calls	Child Calls
cpu_applyFilter	4.449	4.449	24.398.604	0
cpu_gaussian	1.414	3.189	1	8.132.868
cpu_grayscale	0.082	0.082	1	0
cpu_sobel	2.067	4.741	1	16.265.736
freeBMP	0.007	0.007	1	0
get_elapsed	0	0	3	0
main	0.007	9.687	1	11
readBMP	0.354	0.354	1	0
serie	0.001	9.688	1	1
store_result	0	1.307	3	3
writeBMPGrayscale	1.306	1.306	3	0

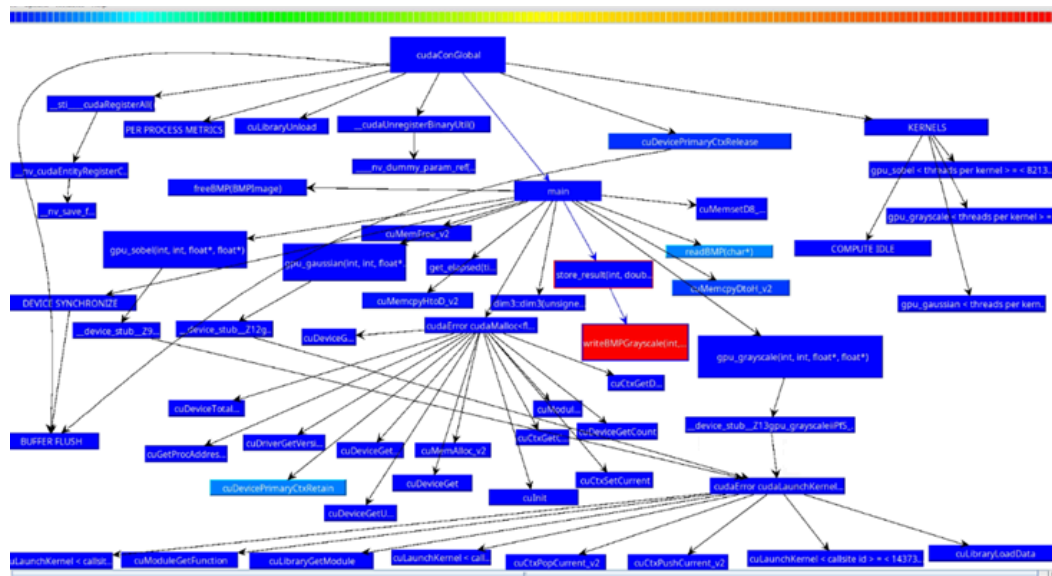
Con los resultados obtenidos fue posible observar que `cpu_applyFilter`, `cpu_sobel`, `cpu_gaussian` y `writeBMPGrayscale` los que más afectan el consumo de tiempo.

Esto tiene sentido ya que son las funciones que realizan la mayor parte del cómputo por cada pixel de la imagen. Por ejemplo `cpu_gaussian` llama a `applyFilter` una vez por cada pixel y `cpu_sobel` lo hace dos veces por cada pixel y ya de por sí `applyFilter` por cada pixel evalúa la matriz 3x3 de pixeles que lo tienen como centro (excepto los bordes).

Implementaciones

- OpenMP:





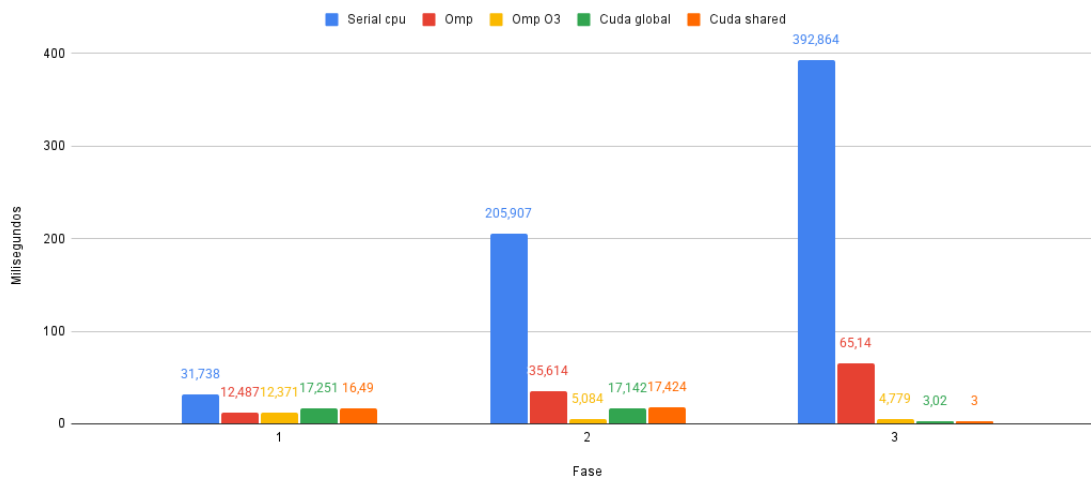
En este caso es writeBMPGrayscale quien consume más tiempo de ejecución

Resultados del procesamiento de imágenes

Ahora se puede ver la comparación de las diferentes implementaciones; siendo el código base de cpu compilado en 3 versiones diferentes, la original (serial) con nvcc, la paralelizada con openmp usando gcc y la última usando la bandera -O3 de optimización.

En Cuda se tienen dos implementaciones, memoria global y memoria compartida. En esta primera comparativa los tiempos de ejecución fueron medidos usando Real de la librería Time, por ende los de Cuda no son enteramente representativos del tiempo de ejecución pero sí del real, ya que no distinguimos entre tiempo de copia (desde ram a la gpu y viceversa) y de kernel (procesamiento).

Tiempo procesamiento imágenes



Las fases son las etapas del procesamiento de la imagen.

Fase 1 es el pasaje a escala de grises.

Fase 2 es el primer filtro de tipo Gaussiano para suavizar la imagen y reducir el ruido, aplicando una función de distribución gaussiana para ponderar los píxeles vecinos.

Fase 3 es el segundo y último filtro de tipo Sobel que busca cambios pronunciados en la primera derivada de una imagen.

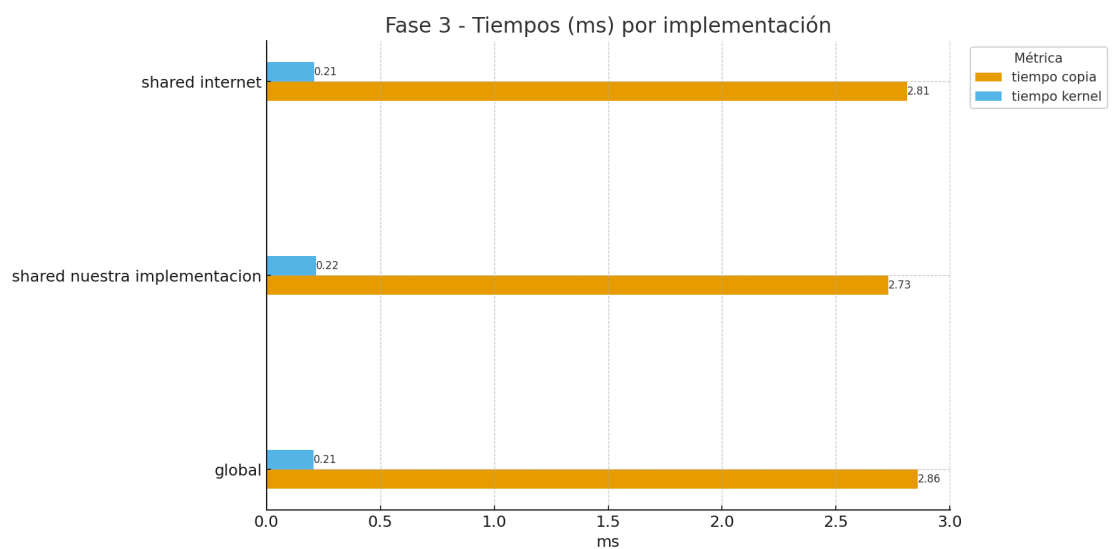
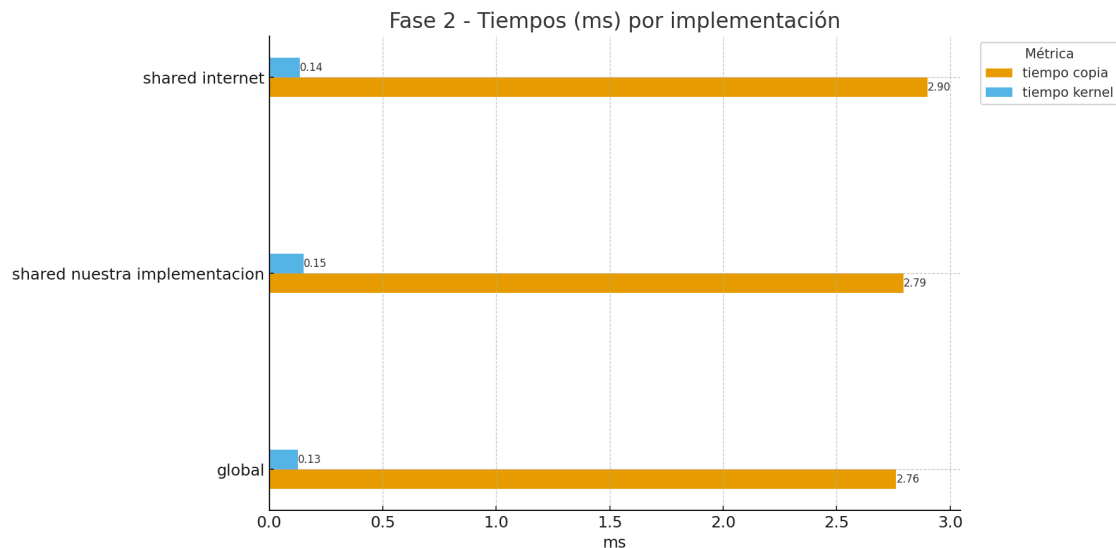
Se puede observar cómo mejora exponencialmente el tiempo de procesamiento usando paralelismo. Lo interesante es el efecto que tiene activar la optimización o3 del compilador en el caso de la cpu.

También se puede apreciar que los tiempo no mejoran mucho usando la aceleración por gpu con Cuda, incluso usando una imagen panorámica muy grande de 150mb no pudimos apreciar mejoría notable, pero esto tiene una explicación, usando el profiler NCU de Nvidia y viendo individualmente los tiempos de carga hacia la memoria de gpu y kernel de procesamiento.

Se encontraron dos resoluciones como forks del proyecto original y también se las agrego para comparar los resultados con la nuestra.

Se hará hincapié en las fases 2 y 3 ya que la primera se comparte de igual manera entre todas las implementaciones.

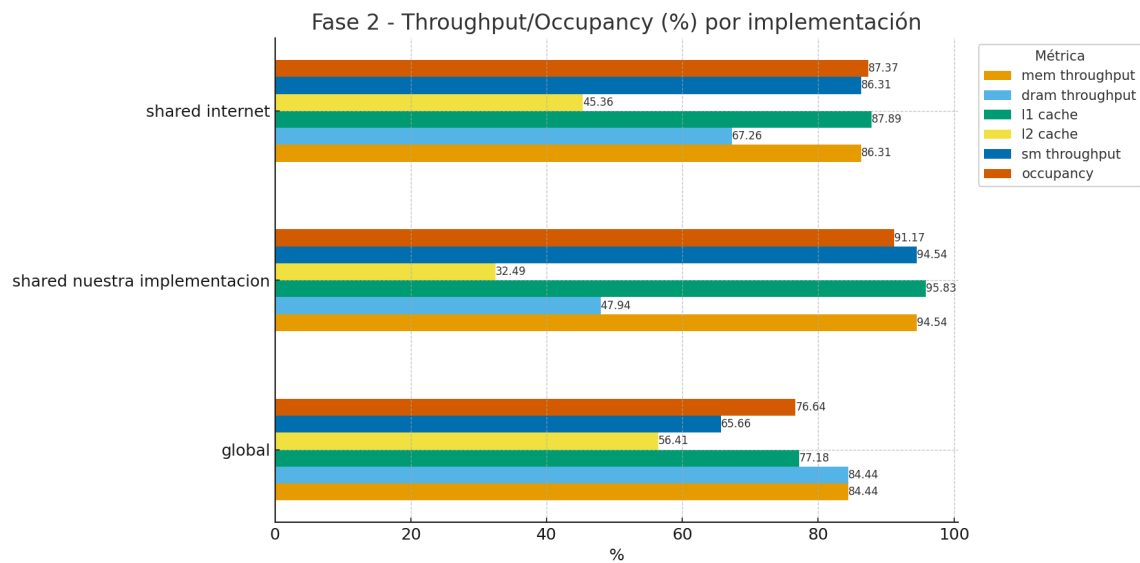
Estos tiempos son medidos usando Cuda event, no son comparables con la implementación de cpu, ya que nos muestran solo el tiempo del kernel en la gpu.



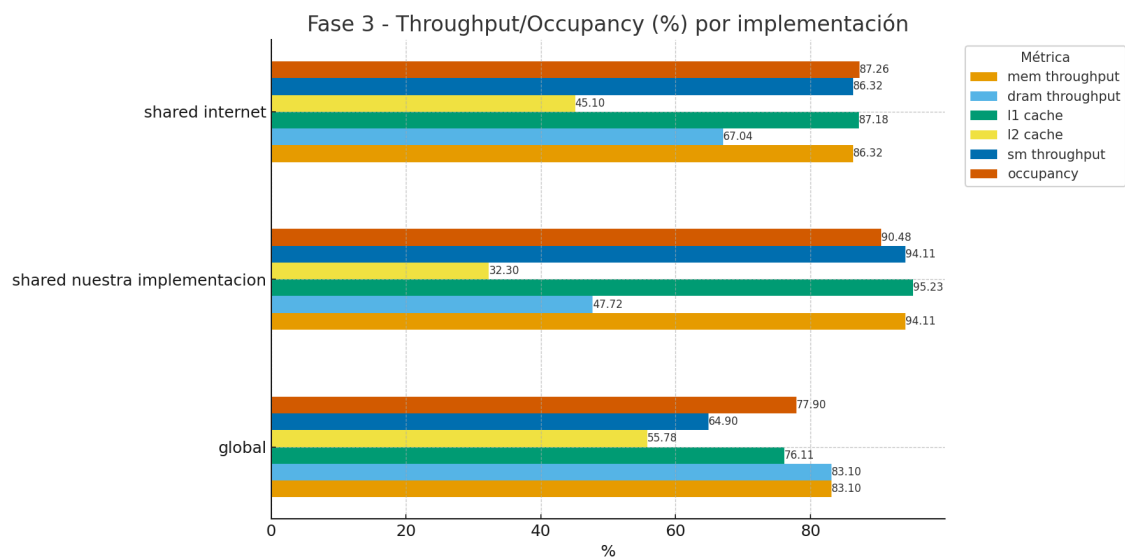
Claramente se nota que el cuello de botella es la copia de datos hacia la gpu y su vuelta a ram de sistema, ya que el tiempo de procesamiento es muy bajo, debajo de medio milisegundo. Es por esto que la implementación del openmp con 03 es tan rápida en comparación, no tiene el overhead de copiar los datos hacia otro dispositivo.

Surgió la pregunta si nuestra implementación del memoria compartida estaba funcionando bien ya que no veíamos mejora, pero a continuación gracias al profiler se puede ver que se están de hecho usando más recursos y aprovechando mejor la gpu pero debido a nuestra tarea es tan pequeña no se ve reflejado en el tiempo de

procesamiento, ya que nuestro overhead es muy grande. Seguramente se vería una mejora si fuese un video a 30, 60 o 240 fps, pero siendo una sola imagen no.



El profiler nos muestra que efectivamente usando memoria compartida logramos más ocupación general de la gpu que con la memoria global, pasando de 83% a 94% y casi 96% de caché L1.



En conclusión se logró hacer uso de la memoria compartida para explotar al máximo la aceleración por gpu con Cuda aunque en este caso específico no se haya visto la mejoría en tiempo de procesamiento.

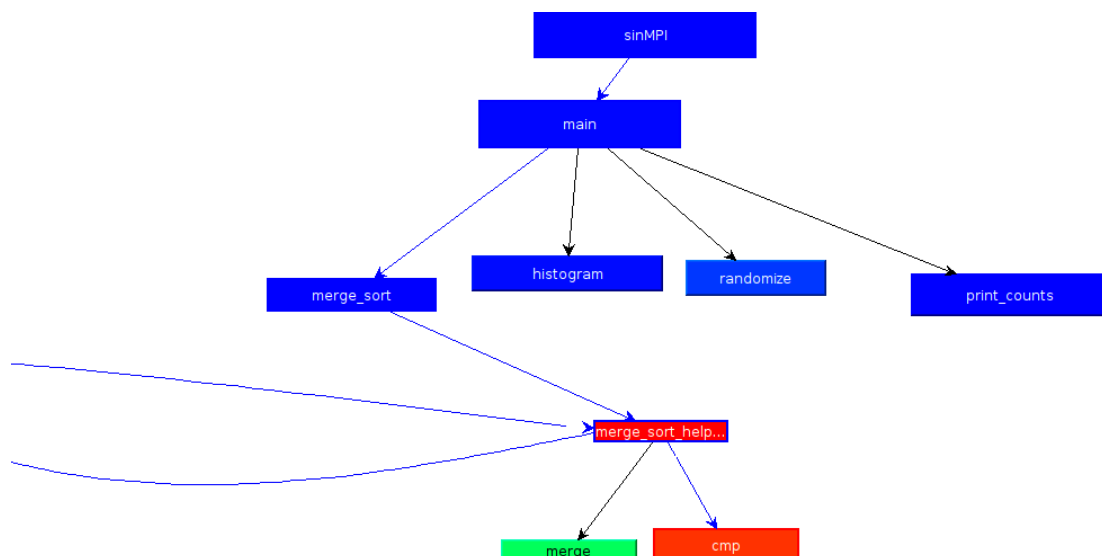
PARTE 2: Paralelismo de MergeSort con MPI

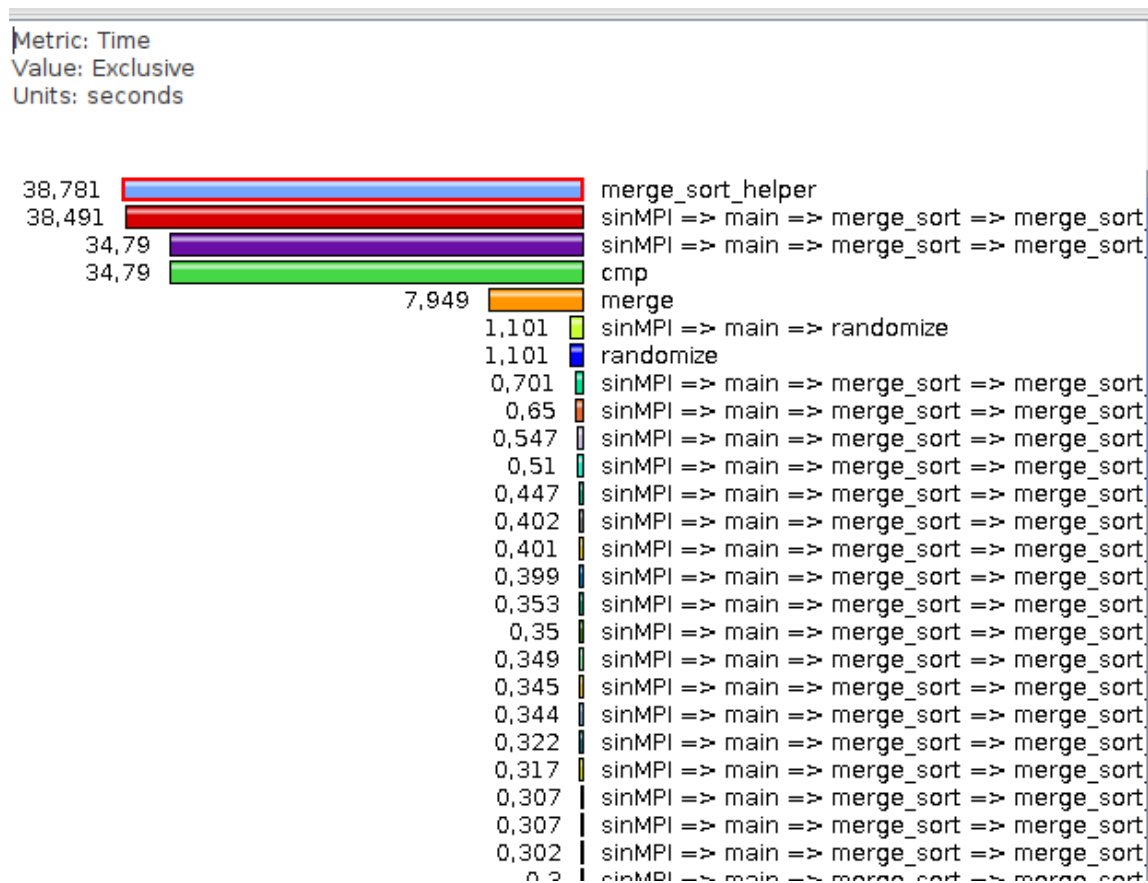
Desarrollo

En esta parte se paraleliza un código que utiliza el algoritmo de mergesort, el cual genera una lista de números enteros aleatorios, los analiza y ordena, para luego reportar cuánto tiempo le tomó ejecutarlo.

La cantidad total de números enteros aleatorios que el programa generará se llamará n . El programa originalmente para un argumento de $n = 100000000$ tarda alrededor de **10,3 segundos** en completarse.

Se procede a realizar el profiling del programa para ver que se podría mejorar.

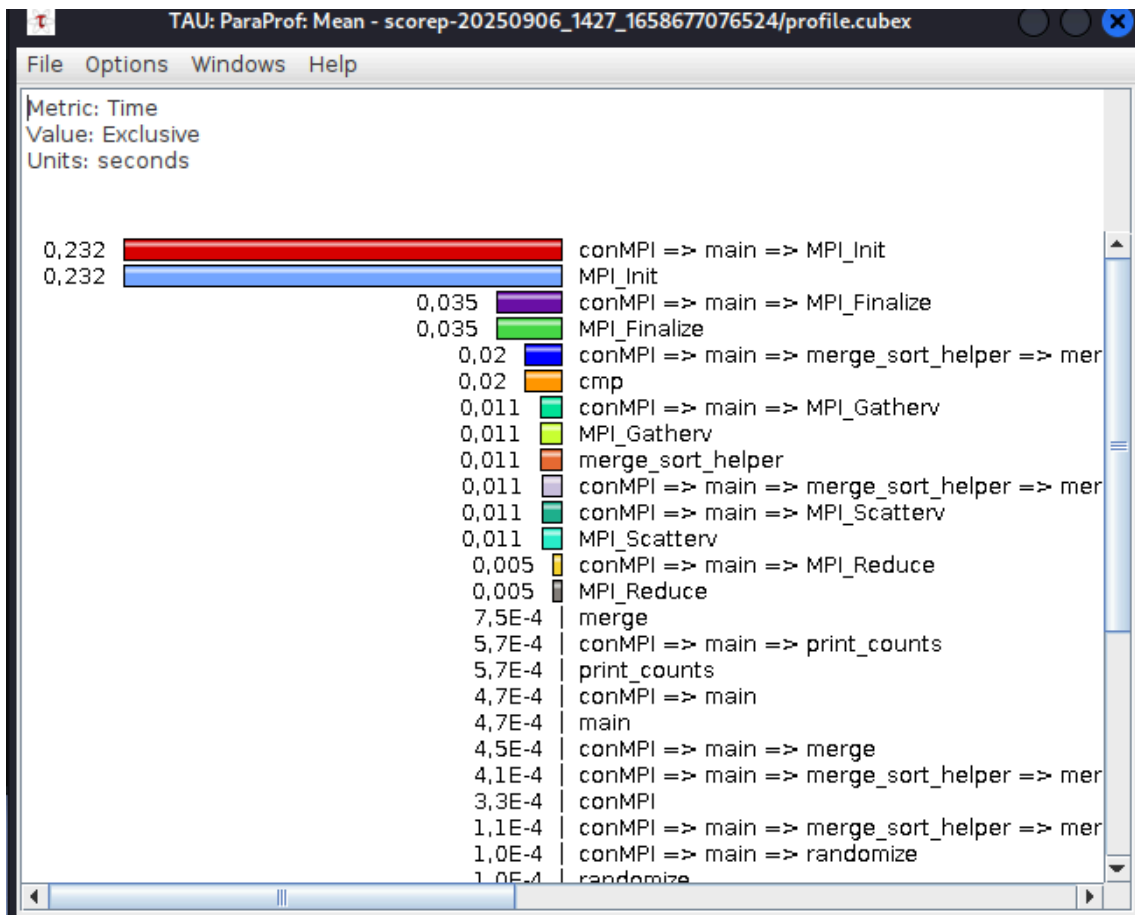
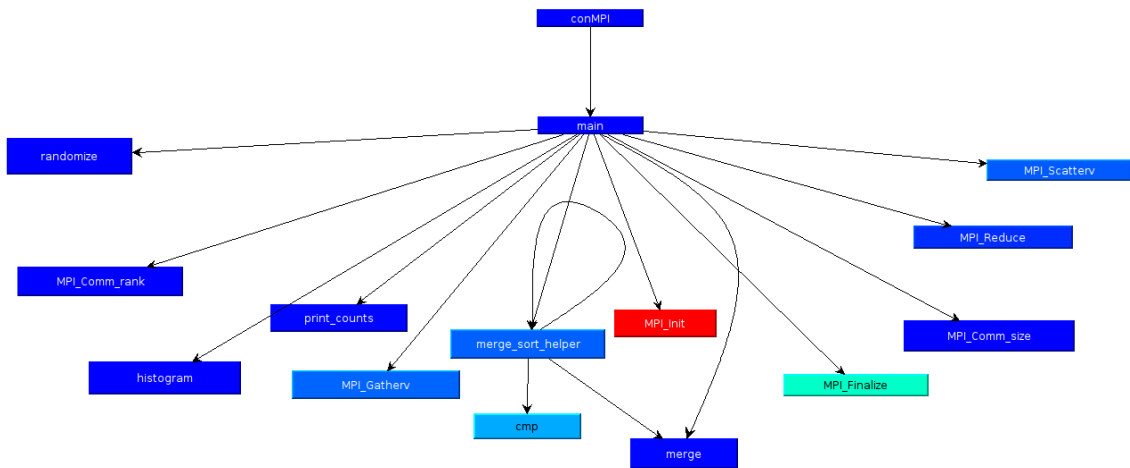




Para mejorar esto usaremos MPI (Message Passing Interface), un estándar de comunicación para programación paralela distribuida que permite que múltiples procesos que corren en una o varias máquinas cooperen intercambiando mensajes (en vez de compartir memoria).

El modo de trabajo consiste en particionar los datos de manera balanceada entre la cantidad de hilos o trabajadores que tengamos disponibles y enviarle esa porción a cada uno para que lo ejecute.

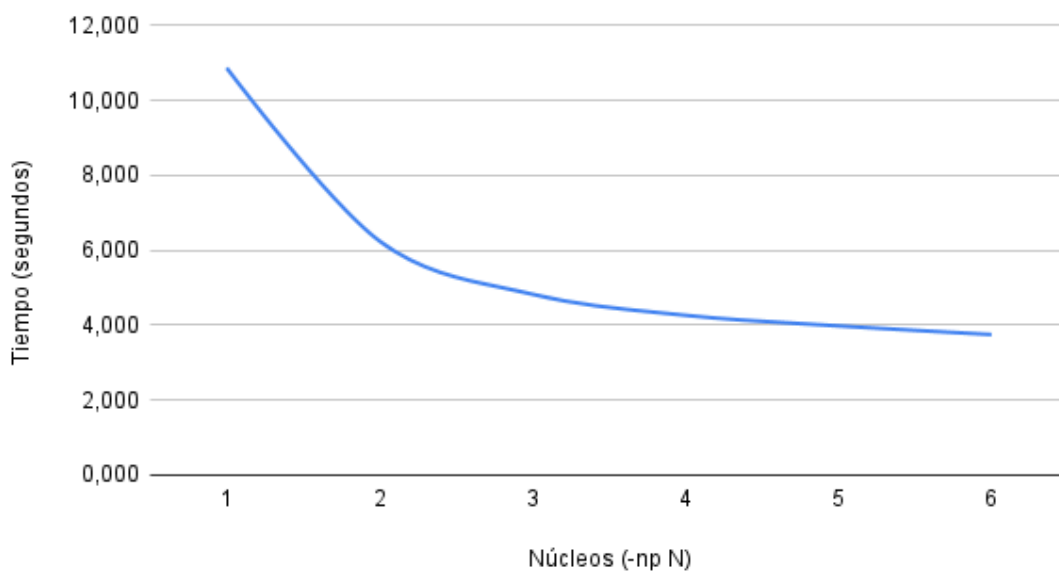
El proceso maestro crea la lista completa, la reparte entre los procesos con **MPI_Scatterv** y cada proceso calcula su histograma local y ordena su bloque. Luego, los histogramas se combinan en uno global mediante **MPI_Reduce**, y las listas ordenadas se reúnen en el maestro con **MPI_Gatherv**. Finalmente, el maestro realiza un merge secuencial de los bloques recolectados para obtener la lista global ordenada y muestra el histograma total junto con los tiempos de ejecución. Se vuelve a realizar un profiling para ver si hubo cambios en los resultados:



Resultados de mergesort

Cantidad de valores	100000000
MPI	
Núcleos (-np N)	Tiempo (usando real de time) (segundos)
1	10,871
2	6,246
3	4,827
4	4,265
5	3,983
6	3,752
Implementación original sin MPI	10,296

Tiempo según núcleos (para arg = 100000000)



Se ve una mejora importante hasta 4 núcleos trabajando en paralelo, luego los resultados no mejoran significativamente. Esto lo podemos explicar en que el cuello

de botella se vuelve el núcleo maestro ya que al final debe unir todas porciones de datos que fueron recogidas, seguramente podemos paralizar esto haciendo fusiones distribuidas y comunicaciones en árbol, pero el código actual la resuelve centralizadamente para simplificar.

Conclusión

Consideramos muy importante todo lo aprendido en el curso principalmente porque fueron temas que no habíamos tocado hasta el momento, y ver cómo podemos acelerar la ejecución de tareas mediante la paralelización nos resultará muy útil para proyectos futuros. Así como el realizar análisis con un profiler y entender cuales son los cuellos de botella en un código y como moldearlos a una posible solución. Especialmente el uso de Cuda para realizar cómputo en grandes cantidades de datos y OpenMP que es algo que seguramente haremos su uso cotidianamente desde ahora para acelerar la ejecución de bucles for.